



10. Datos persistentes entre escenas

Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

Fernando Madrid Recio

Datos entre Escenas

Al cambiar de escena, Unity **destruye todos los GameObjects** de la escena anterior, incluyendo el Player.
¿Cómo trasladar posición, vida o puntuación a la nueva instancia?

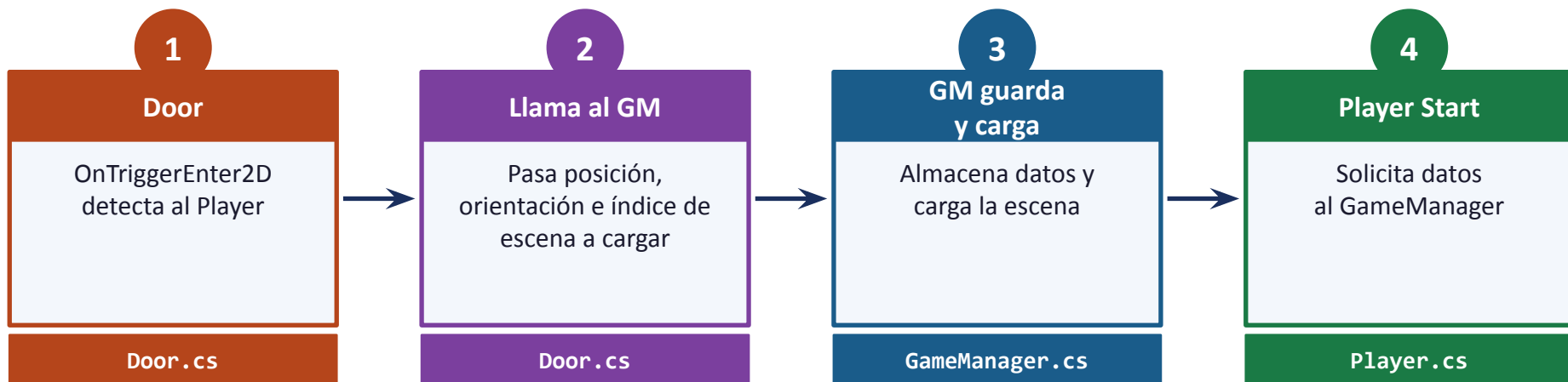
✗ Sin patrón

- El Player guarda sus datos en sus propios scripts.
- Al cargar la escena, el Player viejo es **destruido**.
- El nuevo Player nace **sin** memoria del anterior.
- Los datos se pierden.

✓ Con Singleton GameManager

- Un objeto persistente guarda los datos **entre escenas**.
- Al cambiar de escena los datos **sobreviven** en el GM.
- El nuevo Player pide sus datos al GM en Start().
- Patrón escalable a **cualquier dato** del jugador.

Visión General del Patrón



El Singleton GameManager

El **Singleton** garantiza una **única instancia global** de la clase. Con **DontDestroyOnLoad()** el objeto persiste al cambiar de escena.

```
public class GameManager : MonoBehaviour
{
    // Única instancia accesible globalmente
    public static GameManager Instance;

    // Datos del jugador que viajan entre escenas
    public Vector3 InitPlayerPosition {get; private set;}
    public Vector3 InitPlayerRotation {get; private set;}

    void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else Destroy(gameObject); // destruir duplicados
    }
}
```

Comunicación de Door con GameManager

```
[SerializeField]private Vector3 nextScenePosition;  
[SerializeField]private Vector3 nextSceneOrientation;  
[SerializeField]private int nextSceneIndex;  
  
void OnTriggerEnter2D(Collider2D col)  
{  
    if (col.CompareTag("Player"))  
    {  
        GameManager.Instance.LoadScene(nextScenePosition,nextSceneOrientation,nextSceneIndex);  
    }  
}
```

GameManager guarda los datos y carga la escena

- Recibe posición, rotación e índice de escena de Door.
- Los almacena como campos del GM (que persiste entre escenas).
- Llama a SceneManager.LoadScene() con el índice recibido.
- La escena anterior se destruye; los datos quedan en el GM.

```
// En GameManager.cs
public void LoadNewScene(
    Vector3 newPosition,
    Vector3 newRotation,
    int newSceneIndex)
{
    // 1. Guardar datos
    InitPlayerPosition = newPosition;
    InitPlayerRotation = newRotation;

    // 2. Cargar nueva escena
    SceneManager.LoadScene(newSceneIndex);
}
```

Player solicita sus datos al GameManager

```
// En Player.cs
void Start()
{
    // Leer posición y rotación guardadas
    transform.position = GameManager.Instance.InitPlayerPosition;

    transform.eulerAngles = GameManager.Instance.InitPlayerRotation;
}
```

- Cada instancia de Player nace sin memoria del anterior.
- En Start() **consulta al GM** la posición y orientación guardadas.
- Aplica esos valores a su transform
- Si es la escena inicial, usa los valores **por defecto** del GM.

Extendiendo el Patrón: clase PlayerData

El patrón funciona para cualquier dato del jugador. Es preferible encapsular todo en una **clase contenedora PlayerData** en lugar de añadir campos sueltos al GameManager.

PlayerData.cs

```
// No hereda de MonoBehaviour
public class PlayerData
{
    public Vector3 position;
    public Vector3 rotation;
    public int health = 100;
    public int score = 0;
    public int lives = 3;
    // Añade aquí más campos
}
```

GameManager.cs

```
// Un solo campo en lugar de sueltos
public PlayerData playerData = new PlayerData();

public void LoadNewScene(PlayerData data, int sceneIndex)
{
    playerData = data; // Se actualizan los datos.
    SceneManager.LoadScene(sceneIndex);
}
```

PlayerData: Uso desde Door y Player

Door.cs — rellena el objeto PlayerData:

```
void OnTriggerEnter2D(Collider2D col)
{
    if (col.TryGetComponent(out Stats playerStats)
    {
        PlayerData data = new PlayerData();
        data.position = nextScenePosition;
        data.rotation = nextSceneOrientation;

        // Estado del Player viejo
        data.health = playerStats.health;
        data.score = playerStats.score;

        GameManager.Instance.LoadScene(data,nextSceneIndex);
    }
}
```

Player.cs — lee el objeto PlayerData:

```
void Start()
{
    PlayerData data = GameManager.Instance.playerData;

    // Posición y orientación
    transform.position = data.position;
    transform.eulerAngles = data.rotation;

    // Estado del jugador
    Stats stats = GetComponent<Stats>();
    stats.health = data.health;
    stats.score = data.score;
}
```